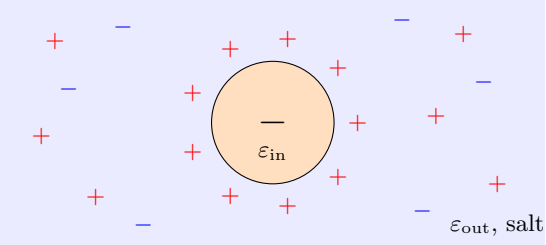
Poisson–Boltzmann equation in 2D



1 Introduction

The Poisson–Boltzmann (PB) equation [1, 2] describes the electrostatic potential around a charged molecule (a protein, DNA) immersed in a salt solution: mobile ions accumulate around the molecule and screen its charge. In reduced units ($\phi = e\psi/k_BT$) it reads

$$-\nabla \cdot (\varepsilon \nabla \phi) + \varepsilon_{\text{out}} \kappa^2 H(\mathbf{x}) \sinh \phi = 4\pi \ell_B \sum_i q_i \, \delta(\mathbf{x} - \mathbf{x}_i),$$

where ε jumps from $\varepsilon_{\text{in}} \approx 2$ inside the molecule to $\varepsilon_{\text{out}} \approx 80$ in water, $H = 0$ inside the molecule (no ions) and $H = 1$ outside, and κ is the inverse Debye length, which grows with the salt concentration. For small potentials $\sinh \phi \approx \phi$ and one obtains the linearized PB equation (LPB); for highly charged molecules such as DNA the linearization overestimates the potential. For an infinitely long charged rod (a crude model of DNA) nothing depends on the rod axis, so the problem is exactly two-dimensional. Production solvers such as NextGenPB [1] solve the 3D problem on adaptive meshes with MPI.

2 Objectives

Write a parallel code using MPI or OpenMP for the 2D PB equation on a square domain with a uniform grid, finite differences, and zero Dirichlet boundary conditions. The molecule is a union of disks with point charges. First solve LPB with the conjugate gradient method with a Jacobi preconditioner (PCG, Section 3), and check the correctness of the code against the analytic solution for a single charged disk, which is given in terms of the modified Bessel functions K_0 and K_1. Then solve the nonlinear PB equation [2] with Newton's method [3], which uses the same PCG solver at each step. Compare LPB and NLPB for a DNA-like rod at different salt concentrations. Run strong and weak scalability tests and report the results.

3 Numerical method

A good reference for everything in this section is the book by Saad [4] (freely available): Sections 6.7 (CG), 9.2 (PCG) and 10.2 (Jacobi and SSOR preconditioners).

Discretization. Take a uniform grid with spacing h and unknowns ϕ_{ij} at the interior nodes ($\phi = 0$ on the boundary). The operator $-\nabla \cdot (\varepsilon \nabla \phi)$ is discretized with the five-point stencil

$$(L\phi)_{ij} = \frac{1}{h^2}\Big[\varepsilon_{i+\frac{1}{2},j}(\phi_{ij} - \phi_{i+1,j}) + \varepsilon_{i-\frac{1}{2},j}(\phi_{ij} - \phi_{i-1,j}) + \varepsilon_{i,j+\frac{1}{2}}(\phi_{ij} - \phi_{i,j+1}) + \varepsilon_{i,j-\frac{1}{2}}(\phi_{ij} - \phi_{i,j-1})\Big],$$

where ε at the midpoints is the harmonic mean of the nodal values, e.g. $\varepsilon_{i+\frac{1}{2},j} = 2\varepsilon_{ij}\varepsilon_{i+1,j}/(\varepsilon_{ij} + \varepsilon_{i+1,j})$, which handles the jump of ε across the molecular surface. Let $D = \varepsilon_{\text{out}}\kappa^2 \,\text{diag}(H_{ij})$, and assign each point charge to the nearest grid node, $f_{ij} = 4\pi\ell_B q/h^2$. The discrete LPB is then the

linear system $(L + D)\phi = f$, and the matrix $L + D$ is symmetric positive definite. It does not need to be stored: the matrix-vector product can be computed on the fly from the arrays ε_{ij} and H_{ij} (matrix-free). With MPI, the grid is split among the processes (in strips or blocks), each matrix-vector product requires exchanging one layer of ghost nodes with the neighbouring processes, and each dot product requires a global reduction (`MPI_Allreduce`).

Linear PB: preconditioned CG. Let $A = L + D$. The number of CG iterations grows with the condition number of A, which here grows like h^{-2} and with the ratio $\varepsilon_{\text{out}}/\varepsilon_{\text{in}}$. A preconditioner $M \approx A$, easy to invert, reduces the number of iterations. The simplest choice is the Jacobi preconditioner $M = \operatorname{diag}(A)$, so that applying M^{-1} is a pointwise division, trivially parallel. It compensates the different scaling of the rows due to the jump in ε, although the iteration count still grows as the grid is refined. Algorithm 1 shows PCG for an SPD system $A\mathbf{x} = \mathbf{b}$; each iteration costs one matrix-vector product, one application of M^{-1}, two dot products and three vector updates. A suitable initial guess is $\mathbf{x}_0 = \mathbf{0}$, or the previous solution when solving a sequence of similar systems.

Algorithm 1 Preconditioned conjugate gradient [4, Algorithm 9.1]

$\mathbf{r}_0 = \mathbf{b} - A\mathbf{x}_0, \quad \mathbf{z}_0 = M^{-1}\mathbf{r}_0, \quad \mathbf{p}_0 = \mathbf{z}_0$
for $k = 0, 1, \ldots$ until $\|\mathbf{r}_k\| \le \text{tol}\,\|\mathbf{b}\|$ **do**
  $\alpha_k = (\mathbf{r}_k, \mathbf{z}_k)/(A\mathbf{p}_k, \mathbf{p}_k)$
  $\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{p}_k$
  $\mathbf{r}_{k+1} = \mathbf{r}_k - \alpha_k A\mathbf{p}_k$
  $\mathbf{z}_{k+1} = M^{-1}\mathbf{r}_{k+1}$
  $\beta_k = (\mathbf{r}_{k+1}, \mathbf{z}_{k+1})/(\mathbf{r}_k, \mathbf{z}_k)$
  $\mathbf{p}_{k+1} = \mathbf{z}_{k+1} + \beta_k \mathbf{p}_k$
end for

Nonlinear PB: Newton's method. The discrete NLPB reads $F(\phi) = L\phi + D\sinh\phi - f = 0$, with sinh applied componentwise. Its Jacobian

$$J(\phi) = L + D\,\operatorname{diag}(\cosh\phi_{ij})$$

has the same structure as the LPB matrix (with $\kappa^2 H$ replaced by $\kappa^2 H\cosh\phi$) and is still SPD, since $\cosh\phi \ge 1$: the same matrix-free product and the same PCG can be reused. Newton's method reads:

1. start from the LPB solution, $\phi^0 = (L + D)^{-1} f$;
2. for $n = 0, 1, \ldots$: solve $J(\phi^n)\,\delta^n = -F(\phi^n)$ with PCG and update $\phi^{n+1} = \phi^n + \lambda_n \delta^n$;
3. stop when $\|F(\phi^{n+1})\| \le \text{tol}\,\|f\|$.

Since sinh grows exponentially, a full step ($\lambda_n = 1$) can overshoot: use a damped step, halving $\lambda_n = 1, \frac{1}{2}, \frac{1}{4}, \ldots$ until $\|F(\phi^{n+1})\| < \|F(\phi^n)\|$. The PCG tolerance can be loose in the first Newton iterations and tightened as $\|F\|$ decreases (inexact Newton). Note that the potential is very large only close to the point charges, which lie inside the molecule where $H = 0$, so the nonlinear term only acts in the solvent.

4 Ideas for an exam project

- Size-modified PB (SMPB) with mixed salts (e.g. Na^+ and Mg^{2+}), to account for the finite size of ions [3]
- Compute the interaction energy between two charged rods as a function of their distance
- A better preconditioner than Jacobi (SSOR [4], multigrid) or an adaptive (quadtree) grid

References

[1] Di Florio, V., Ansalone, P., Siryk, S. V., Decherchi, S., de Falco, C., & Rocchia, W. (2025). NextGenPB: an analytically-enabled super resolution tool for solving the Poisson–Boltzmann equation featuring local (de)refinement. Computer Physics Communications, 317, 109816. (https://doi.org/10.1016/j.cpc.2025.109816, code: https://github.com/concept-lab/NextGenPB)
[2] Li, Z., Pao, C. V., & Qiao, Z. (2007). A finite difference method and analysis for 2D nonlinear Poisson–Boltzmann equations. Journal of Scientific Computing, 30(1), 61–81.
[3] Chaudhry, J. H., Bond, S. D., & Olson, L. N. (2011). Finite element approximation to a finite-size modified Poisson–Boltzmann equation. Journal of Scientific Computing, 47(3), 347–364. (PDF)
[4] Saad, Y. (2003). Iterative Methods for Sparse Linear Systems, 2nd edition. SIAM. (https://www-users.cse.umn.edu/~saad/IterMethBook_2ndEd.pdf)